

13. YOLO & mAP

Overview

YOLO detects objects in a single forward pass by predicting boxes and class scores on a grid. We define IoU, precision to recall, average precision, and mAP, then trace YOLO architectural changes from v3 through v8 and export considerations.

Lab: Lab 13.

Prior modules: Module 12.

Contents

1	Single-Shot Detection	2
2	Grid Formulation	2
3	Loss Function	2
4	Non-Maximum Suppression	2
5	mAP Metric	2
6	YOLO Evolution	3
7	Lab	3
8	Anchor-Free Variants	3
8.1	Export for Edge	3
9	Extended Intuition	3
10	Numerical Walkthrough	4
11	PyTorch Implementation Notes	4
12	Local GPU Constraints	4
13	Debugging Checklist	4
13.1	Assignment, Loss Terms, and Deployment Checks	5
14	Practice Problems	5

Module track

This module builds on **Module 12**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

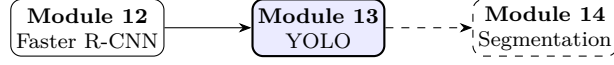


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Single-Shot Detection

YOLO [4] frames detection as regression: one forward pass predicts all boxes. Contrasts with two-stage **Module 12** (Faster R-CNN pipeline).

2 Grid Formulation

Divide image into $S \times S$ grid; each cell predicts B boxes and class probabilities:

$$\hat{\mathbf{y}} = (\mathbf{b}_1, \dots, \mathbf{b}_B, P(c_1), \dots, P(c_C)). \quad (1)$$

Responsible cell: center of ground-truth object falls in that cell.

Note. Unified loss enables real-time inference, critical for video and edge deployment.

3 Loss Function

$$\begin{aligned} \mathcal{L} = & \lambda_{\text{coord}} \sum \mathbb{K}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum \mathbb{K}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\ & + \sum \mathbb{K}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum \mathbb{K}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum \mathbb{K}_i^{\text{obj}} \sum_c (p_i(c) - \hat{p}_i(c))^2. \end{aligned} \quad (2)$$

4 Non-Maximum Suppression

Suppress overlapping predictions keeping highest confidence:

$$\text{Keep box } i \text{ if } \text{IoU}(i, j) < \tau_{\text{nms}} \forall j \text{ with higher score.} \quad (3)$$

Typical $\tau_{\text{nms}} = 0.5$.

5 mAP Metric

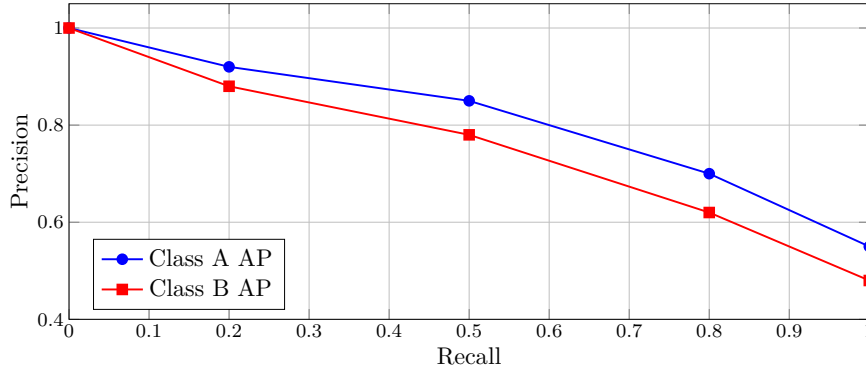
Average Precision per class integrates precision to recall curve:

$$AP_c = \int_0^1 P(R) dR, \quad \text{mAP} = \frac{1}{C} \sum_{c=1}^C AP_c. \quad (4)$$

COCO uses IoU thresholds 0.50:0.05:0.95 (mAP@[.5:.95]).

Table 1: COCO detection metrics.

Metric	Meaning
AP@0.5	Loose localization
AP@0.75	Strict localization
AP _{small}	Objects area $< 32^2$ px

**Figure 2:** Precision to recall curves for AP computation.

6 YOLO Evolution

YOLOv3+ uses multi-scale predictions and anchor boxes akin to **Module 12** (anchor boxes). YOLOv8 adopts anchor-free heads in some variants.

7 Lab

Lab 13 trains YOLO on a detection dataset; report mAP from (5) vs. **Lab 12**.

8 Anchor-Free Variants

Center-ness predicts distance to box center; Focal Loss handles class imbalance:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t. \quad (5)$$

YOLOv5+ uses CSPDarknet backbone with PANet feature fusion.

8.1 Export for Edge

ONNX/TensorRT conversion requires fixed input size and fused NMS ops.

9 Extended Intuition

YOLO treats detection as regression: a single network predicts bounding boxes and class probabilities on a grid in one forward pass. Each grid cell responsible for an object center predicts B boxes (with confidence) and C class scores. Speed comes from no separate proposal stage; accuracy historically lagged two-stage detectors but modern YOLO versions narrowed the gap.

IoU measures box overlap: $\text{IoU} = \text{Area}(A \cap B) / \text{Area}(A \cup B)$. Precision-recall curves per class feed into Average Precision (AP); mAP averages AP across classes at a fixed IoU threshold (often 0.5

or COCO-style 0.5:0.95). Non-maximum suppression (NMS) removes duplicate boxes by keeping highest score boxes and dropping high-IoU neighbors.

Anchor-free YOLO variants (v8+) predict box centers and sizes directly; training still needs careful assignment of ground truth to grid cells. Compare pipeline complexity with Faster R-CNN in **Module 12**: YOLO wins latency, two-stage wins on crowded small objects sometimes.

10 Numerical Walkthrough

YOLOv8n on 640px input, 80 COCO classes, batch $B = 8$.

Output tensor structure (conceptual): for each of $S \times S$ cells, predict box coords, objectness, class logits. If $S = 80$ for 640 input (stride 8), $80^2 = 6400$ cells; each may emit multiple candidates before NMS.

Example box match: GT center falls in cell (i, j) ; assign that cell to predict IoU-best box among anchors or direct coords. Confidence target often $\sqrt{\text{IoU}}$ for matched predictions.

mAP@0.5 computation sketch: for class "person", sort 100 detections by score; compute precision/recall at each rank against 50 GT boxes; AP = area under interpolated PR curve. If model hits 0.45 mAP@0.5 on a small custom dataset after 50 epochs, that is a reasonable lab outcome.

Training lr = 0.01 with cosine decay (Ultralytics default), warmup 3 epochs; loss combines box, cls, dfl terms in v8.

11 PyTorch Implementation Notes

- Ultralytics: `from ultralytics import YOLO`, then `model = YOLO("yolov8n.pt")` and `model.train(data="data.yaml", epochs=50, imgsz=640, batch=8)`.
- `data.yaml` lists `train`, `val` paths and `nc`, `names` for classes.
- Labels in YOLO format: `class x_center y_center width height` normalized to $[0, 1]$ per line per object.
- Export: `model.export(format="onnx")` for deployment; verify opset compatibility.
- Validate: `model.val()` prints mAP50, mAP50-95; use same `imgsz` as training.

```
results = model.train(data="custom.yaml", epochs=50, imgsz=640, batch=8, device=0)
metrics = model.val()
print(metrics.box.map50, metrics.box.map)
```

12 Local GPU Constraints

YOLOv8n at 640px, $B = 8$ fits 4 GB; YOLOv8s may need $B = 4$. Use `cache=True` on small datasets to reduce disk IO; watch RAM on Windows.

Mixed precision enabled by default in Ultralytics on GPU; override with `amp=False` if NaNs on old drivers. For inference on 1650, 640px real-time is achievable with nano/small variants.

13 Debugging Checklist

- mAP zero: labels not converted to YOLO format or wrong class indices (must start at 0).
- Boxes shifted: forgot to normalize coords or confused xyxy vs xywh.
- Thousands of duplicate boxes: NMS IoU threshold too high or confidence threshold too low.
- OOM: reduce `imgsz` to 512 or batch to 4.

- Train loss drops but val mAP flat: train/val domain mismatch or too few val images.

13.1 Assignment, Loss Terms, and Deployment Checks

YOLOv8 decouples the head into box, classification, and distribution focal loss (DFL) branches instead of a single combined tensor like YOLOv3. Box loss uses CIOU or similar to penalize center offset, aspect ratio, and enclosure simultaneously, not just raw IoU. Classification uses BCE with logits per class (multi-label capable) or single-label argmax depending on dataset; verify `data.yaml` class count matches label files. Task assignment in anchor-free YOLO picks the top- k grid cells closest to each GT box center rather than a single responsible cell, which stabilizes training when objects sit near cell boundaries.

DFL treats each box coordinate as a discrete distribution over bins and minimizes cross-entropy, giving finer sub-pixel regression than direct MSE on width/height. Loss weights (`box`, `cls`, `df1`) default in Ultralytics configs; on tiny datasets bump `box` slightly if boxes are jittery while mAP is reasonable. Augmentation pipeline (mosaic, mixup, HSV jitter) is on by default; disable mosaic for the last 10 to 15 epochs (`close_mosaic`) to reduce train-eval domain gap.

Inference path: letterbox resize preserves aspect ratio with gray padding, forward pass, decode boxes from stride-scaled grids, confidence filter, NMS. Letterbox padding must be reversed when mapping predictions back to original image coordinates for evaluation scripts. Export to ONNX at `imgsz=640` then run TensorRT only after verifying NMS is embedded or applied consistently in the deployment graph. FPS benchmarks should fix batch size 1, warm up 10 runs, and report at the same `imgsz` as training; comparing 640 train to 416 deploy inflates latency numbers artificially. When benchmarking against Faster R-CNN (**Module 12**), log preprocessing time separately: YOLO letterbox is cheap, two-stage resize+normalize can add 5 to 10 ms on CPU-bound pipelines. Small object detection below 32px often needs higher input resolution (`imgsz=1280`) or tiled inference; mAP gains come at linear FPS cost on a 1650. Class imbalance in custom datasets: oversample rare-class images in `data.yaml` or increase `cls` loss weight; monitor per-class AP, not mAP alone. Track `results.csv` columns `train/box_loss` and `metrics/mAP50`; divergence (box loss down, mAP flat) signals label noise or val set too small. TensorRT export on Windows may require ONNX opset 12 and fixed batch; dynamic batch engines need separate profiling for batch 1 vs batch 8 latency claims. Albumentations or torchvision transforms must update bbox coords when flipping; YOLO label files do not auto-sync if you preprocess outside Ultralytics pipeline. Confusion matrix per class from `model.val()` helps when mAP hides one missing class entirely on imbalanced custom data. Half-precision inference (`half=True`) doubles FPS on 1650 for YOLOv8n at 640px; validate mAP within 0.5 points of fp32 before deploying. Multi-GPU training in Ultralytics uses DDP automatically when `device=[0,1]`; single 1650 lab stays on `device=0`. Export `results.csv` after each run; plot `metrics/mAP50` vs epoch to catch overfit when train box loss keeps falling. Roboflow or CVAT export to YOLO format saves manual conversion time; still spot-check 5 random labels visually after import. Negative images (no objects) in training reduce false positives; include 10 to 20% empty backgrounds in custom detection datasets. Warmup epochs (`warmup_epochs=3`) stabilize early loss spikes when lr defaults are tuned for full COCO, not 200-image custom sets.

14 Practice Problems

Problem 1. Compute IoU for boxes (10,10,50,50) and (30,30,70,70) in xyxy format.

Problem 2. Convert one COCO xywh absolute annotation to YOLO normalized line for a 640×480 image.

Problem 3. Train YOLOv8n vs Faster R-CNN from **Module 12** on same data; compare mAP and FPS.

Problem 4. Plot PR curve for one class from saved predictions.

Run **Lab 13**; detection foundations from **Module 12**; segmentation in **Module 14**.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 13.

Topic	Primary source	Where in this PDF
YOLO single-pass detection	[4]	Section 1
mAP / IoU definitions	[5]	Eq. 5
Two-stage detector contrast	[1]	Overview

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

- [1] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [3] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, et al. Feature pyramid networks for object detection. In *CVPR*, 2017.
- [4] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016.
- [5] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.